UCS503 – Software Engineering Lab

Public Infrastructure Issue Resolution System

AI-Assisted Triage and Verification – Progress Report

Submitted by:

1024240052 Naitik Garg
1024240047 Shivesh
1024240050 Aditya Raj
1024240064 Lakshya Saraswat

B.Tech, Artificial Intelligence and Machine Learning

Submitted to:

Anushka

Progress Report – 08 September 2026

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Project Selection Phase **6**

1.1 Software Bid 6

1.2 Project Overview 8

1.2.1 Background and Context 8

1.2.2 Problem Statement 8

1.2.3 Project Objectives 8

2 Analysis Phase **9**

2.1 Use Cases 9

2.1.1 Use-Case Diagram 10

2.1.2 Use Case Templates 10

2.2 Activity Diagram and Swimlane Diagram 13

2.3 Data Flow Diagrams (DFDs) 13

2.3.1 DFD Level 0 15

2.3.2 DFD Level 1 15

2.3.3 DFD Level 2 17

2.4 Software Requirement Specification in IEEE Format 17

2.5 User Stories and Story Cards 21

3 Design and Implementation **24**

3.1 System Architecture 24

3.1.1 High-Level Design 24

3.1.2 Technical Stack 24

3.2 Implementation Details 24

3.2.1 Completed Components 26

3.2.2 Remaining Components 26

4 Results and Evaluation **28**

4.1 Testing Methodology 28

4.2 Performance Metrics 28

4.3 Analysis 28

5 Conclusion **29**

5.1 Summary of Work 29

5.2 Key Findings 29

5.3 Future Work and Recommendations 29

5.4 Final Remarks 30

Bibliography **31**

List of Figures

2.1 Use case diagram 10
2.2 Swimlane activity diagram 14
2.3 DFD Level 0 (context diagram) 15
2.4 DFD Level 1 16
2.5 DFD Level 2 (process 3) 17

3.1 Class diagram 25

List of Tables

1.1 Project team 6
1.2 Proposed technology stack 7
1.3 Effort distribution across modules 7

2.1 SRS definitions and acronyms 18
2.2 SRS user classes 19
2.3 Functional requirements 20
2.4 Non-functional requirements 21

3.1 Module-wise completion weighting 26

4.1 Progress snapshot at the current evaluation point 28

Chapter 1

Project Selection Phase

1.1 Software Bid

The team proposes to build the *Public Infrastructure Issue Resolution System*, a role-based complaint-handling workflow for municipal infrastructure that is supported by three advisory AI models for triage.

Team

Roll No.	Name
1024240052	Naitik Garg
1024240047	Shivesh
1024240050	Aditya Raj
1024240064	Lakshya Saraswat

Table 1.1: Project team

The team members are B.Tech students in Artificial Intelligence and Machine Learning. The project is submitted to Anushka, Computer Science and Engineering Department, Thapar Institute of Engineering and Technology, Patiala.

Scope of the Bid

- Citizen reporting with text, photo/video evidence and GPS location, plus tracking until closure.
- Role-based workflow across four stages: intake, review and assignment, repair, and closure.
- Three advisory AI models: complaint classification, severity/priority prediction, and duplicate complaint detection.
- A complete audit trail (`StatusHistory`) of every complaint state change.

- Dashboards for the Citizen, Supervisor, Technician, Inspector and Administrator roles.

Proposed Technology

Layer	Choice
Backend	FastAPI [Ram26]
Authentication	JWT [JBS15] with RBAC middleware [FK92]
Database	Relational database
Frontend	Web dashboard with role-specific views
AI layer	Three advisory models feeding the review stage

Table 1.2: Proposed technology stack

Effort Distribution

Module	Weight
AI Model Integration (3 core models)	20%
Database Connectivity & Storage	15%
Backend API & Business Logic	15%
Login / Authentication Page	10%
Basic Dashboard	10%
UML / SE Design Diagrams	10%
Complaint Data-Flow Implementation	10%
Full Role Dashboards & UI Polish	5%
Testing & Deployment	5%
Total	**100%**

Table 1.3: Effort distribution across modules

Deliverables: a working web application, three integrated AI models, UML and DFD design documents, and a tested deployment.

1.2 Project Overview

1.2.1 Background and Context

Municipal infrastructure — roads, streetlights, water supply, drainage, and sanitation — is a shared civic resource that only stays functional through continuous, complaint-driven repair cycles. Most municipal bodies still handle these repair cycles through manual registers, phone calls, or fragmented spreadsheets maintained separately by each department, which leads to lost complaints and duplicate reports, and leaves citizens with no way of knowing whether their issue was even received.

The Public Infrastructure Issue Resolution System addresses this by giving citizens a single structured channel to report problems — using text, media evidence, and GPS location — and moving each complaint through an auditable, role-based workflow until it is verified as resolved. An internal AI module supports triage and verification, but remains a decision-support component: it produces predictions and recommendations, while human roles (Supervisor, Technician, Inspector, Administrator) remain responsible for all consequential decisions.

1.2.2 Problem Statement

Citizens often lack a reliable, transparent channel to report infrastructure problems and track them to closure, and municipal staff spend significant effort on manual triage — understanding the issue, estimating urgency, identifying duplicates, and deciding departmental ownership. This project's core AI implementation is deliberately scoped to three semester-feasible functions: complaint classification, severity/priority prediction, and duplicate complaint detection, so the system remains a complete information system first and an AI system second.

1.2.3 Project Objectives

1. **Citizen-facing reporting and tracking:** Provide citizens with a simple channel to report issues using text, media, and GPS location, and let them track a complaint from submission through closure.
2. **Core AI-assisted triage:** Implement and evaluate complaint classification, severity/priority prediction, and duplicate complaint detection, keeping their outputs advisory.
3. **Human-in-the-loop governance:** Enforce clear boundaries between Supervisor, Inspector, Technician, and Administrator roles, so AI cannot autonomously approve, reject, assign, or close a complaint.
4. **Full auditability:** Maintain a complete `StatusHistory` of every complaint state transition, attributable to a human user or a system workflow event.

Chapter 2

Analysis Phase

2.1 Use Cases

The system has five human actors — Citizen, Supervisor, Technician, Inspector, and Administrator — and an internal AI Analysis Module that supports the classification, severity/priority prediction, and duplicate-detection use cases.

2.1.1 Use-Case Diagram

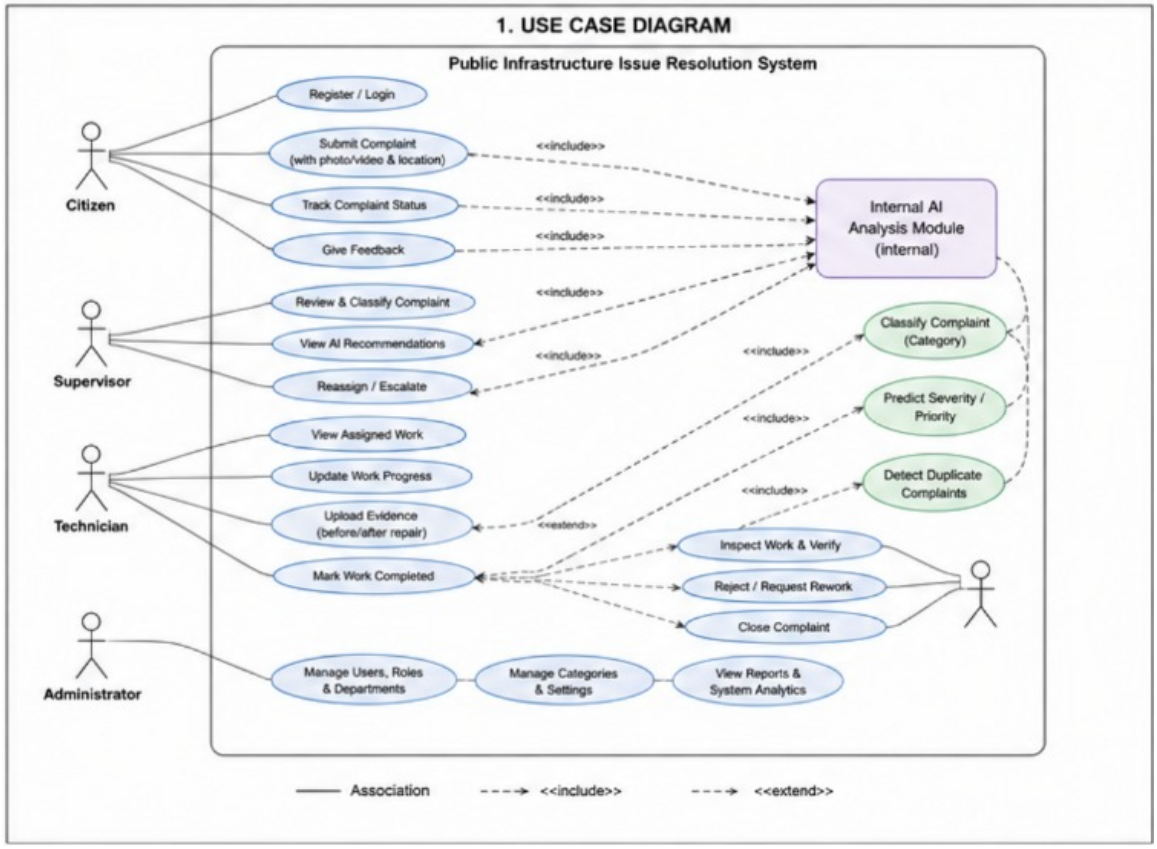


Figure 2.1: Use case diagram for the Public Infrastructure Issue Resolution System, modelled in UML notation [Obj17]. Citizens submit, track, and give feedback on complaints; Supervisors review AI classifications and reassign or escalate work; Technicians act on assigned work and upload repair evidence; an Inspector verifies completed work before closure; and Administrators manage users, categories, and reporting. The internal AI Analysis Module backs the classification, severity/priority prediction, and duplicate-detection use cases via `<<include>>` relationships.

2.1.2 Use Case Templates

The six templates below cover the main path of a complaint from submission to closure, together with administration. Each template maps to a use case in Figure 2.1.

UC-01: Submit Complaint

Primary actor	Citizen
Supporting actor	Internal AI Analysis Module
Description	Citizen reports an infrastructure issue with text, photo/video and GPS location.
Precondition	Citizen is registered and logged in.
Main flow	1. Citizen opens the complaint form. 2. Enters title and description. 3. Attaches photo or video evidence. 4. Shares GPS location. 5. Submits. 6. System validates the input and stores the complaint with status `New`. 7. AI module predicts category, severity/priority and duplicate score, and stores an `AIAnalysis` record. 8. Citizen receives a notification confirming submission.
Alternate flow	6a. Input is invalid or location is missing: the system asks the citizen to correct it.
Postcondition	Complaint is stored with status `New`, with an `AIAnalysis` record, and is visible to Supervisors.

UC-02: Review and Classify Complaint

Primary actor	Supervisor
Description	Supervisor reviews the AI recommendations and confirms category, priority and ownership.
Precondition	Complaint exists with status `New` and an `AIAnalysis` record.
Main flow	1. Supervisor opens the complaint. 2. Views AI category, severity, priority score and duplicate flag. 3. Confirms or corrects the classification. 4. Assigns a Department and Technician. 5. System creates a `WorkOrder` and sets complaint status to `Assigned`. 6. System logs the change in `StatusHistory` and notifies the Technician.
Alternate flow	3a. AI flags a likely duplicate: Supervisor links it to the original complaint. 4a. Wrong department: Supervisor reassigns or escalates.
Postcondition	Complaint has a confirmed category and an active `WorkOrder`.
Business rule	AI output is advisory only; it cannot approve, reject, assign or close a complaint.

UC-03: Update Work Progress and Upload Evidence

Primary actor	Technician
Description	Technician works on the assigned repair and uploads before/after evidence.
Precondition	A `WorkOrder` is assigned to the Technician.
Main flow	1. Technician views assigned work. 2. Sets `WorkOrder` status to `InProgress`. 3. Uploads before-repair evidence. 4. Carries out the repair. 5. Uploads after-repair evidence and adds notes. 6. Marks work as `Completed`. 7. Complaint status becomes `PendingVerification`.
Postcondition	Complaint awaits Inspector verification.

UC-04: Inspect Work and Verify

Primary actor	Inspector
Description	Inspector checks completed work before the complaint can close.
Precondition	Complaint status is `PendingVerification`.
Main flow	1. Inspector opens the completed work order. 2. Reviews before/after evidence and notes. 3. Approves the work. 4. Complaint status becomes `Resolved`. 5. Change is logged in `StatusHistory`.
Alternate flow	3a. Work is unsatisfactory: Inspector requests rework; `WorkOrder` status becomes `ReworkRequested` and returns to the Technician.
Postcondition	Complaint is `Resolved` or sent back for rework.

UC-05: Close Complaint and Give Feedback

Primary actors	Inspector (close), Citizen (feedback)
Precondition	Complaint status is `Resolved`.
Main flow	1. Inspector closes the complaint. 2. Status becomes `Closed` and is logged. 3. Citizen is notified. 4. Citizen rates the resolution from 1 to 5 and adds comments. 5. System stores the `Feedback` record.
Postcondition	Complaint is `Closed` with an audit trail and optional feedback.

UC-06: Manage Users, Roles and Departments

Primary actor	Administrator
Description	Administrator maintains users, roles, departments, categories and settings, and views reports.
Precondition	Administrator is logged in.
Main flow	1. Administrator opens the admin panel. 2. Adds, edits or deactivates users and sets their roles. 3. Manages departments and complaint categories. 4. Views reports and system analytics.
Postcondition	User, role and category data is updated.

2.2 Activity Diagram and Swimlane Diagram

Figure 2.2 shows the full complaint lifecycle as a single swimlane activity diagram, with one lane per participant. The two decision points are the Supervisor's duplicate check and the Inspector's verification; unsatisfactory work loops back to the Technician as rework. The highlighted AI step only produces recommendations.

2.3 Data Flow Diagrams (DFDs)

The data-flow design follows structured-analysis notation [You89]: Level 0 shows the system as a single process, Level 1 breaks it into the four workflow stages, and Level 2 decomposes the AI analysis process.

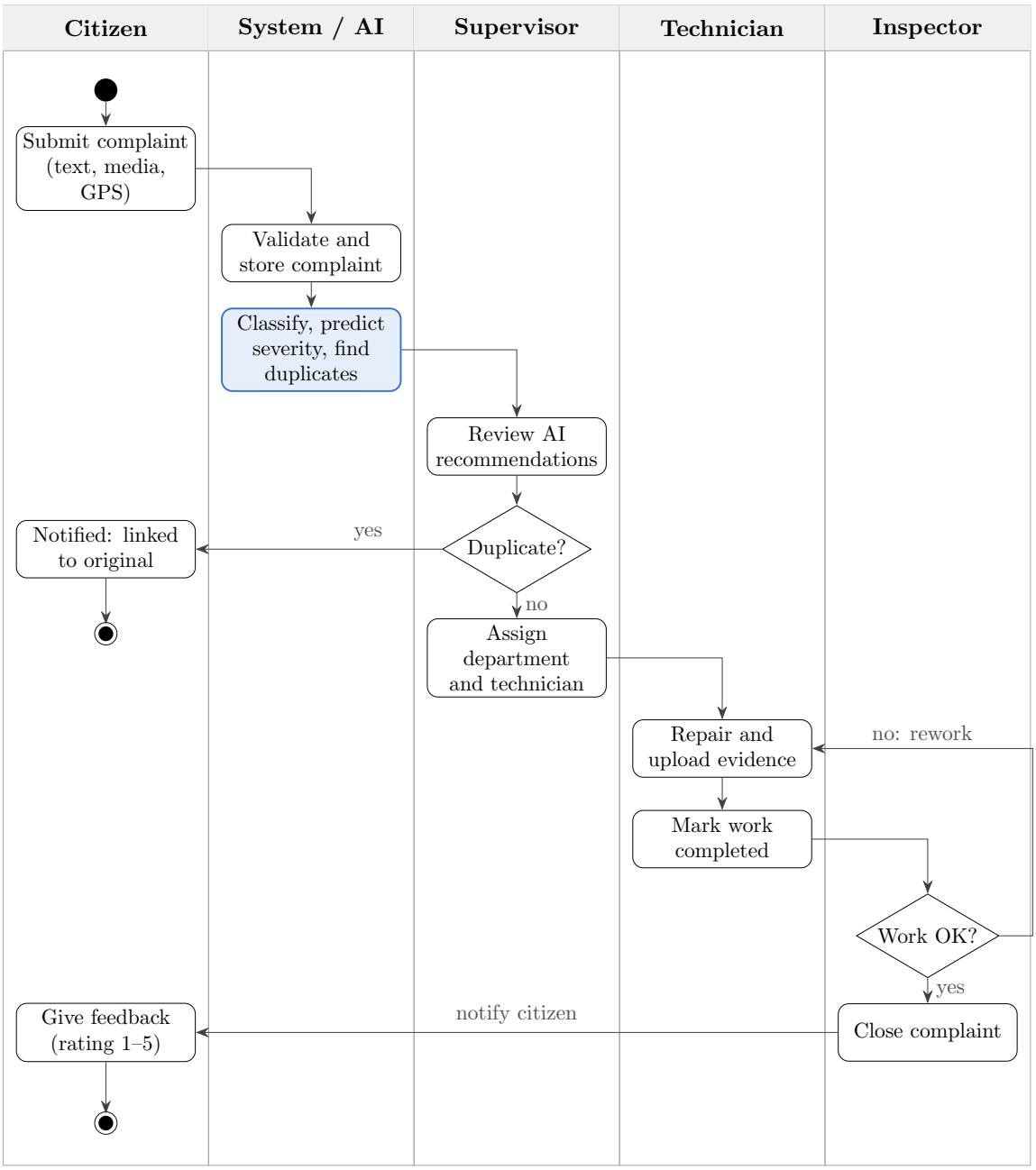


Figure 2.2: Swimlane activity diagram of the complaint lifecycle across the Citizen, System/AI, Supervisor, Technician and Inspector lanes. A complaint closes only after Inspector approval.

2.3.1 DFD Level 0

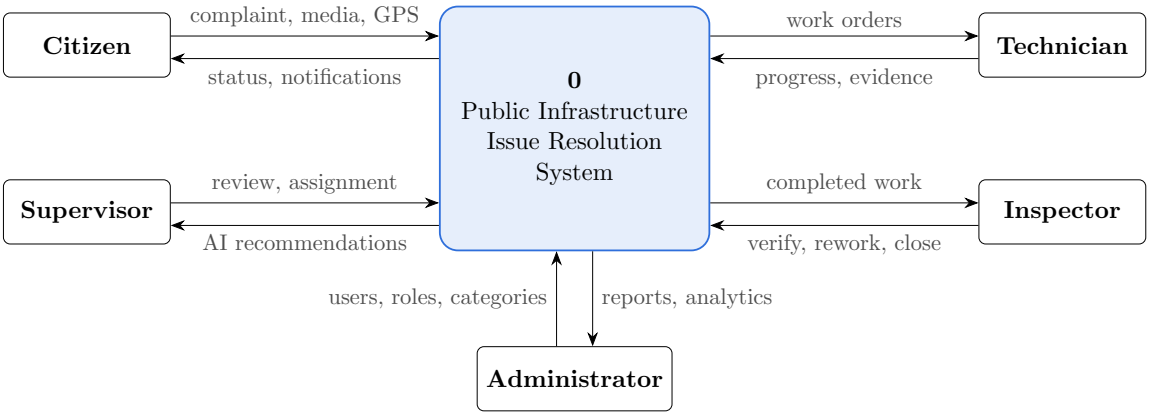


Figure 2.3: DFD Level 0 — context diagram. The whole system is one process (0); each role is an external entity that sends inputs to and receives outputs from it.

2.3.2 DFD Level 1

Figure 2.4 expands process 0 into fourteen processes grouped in four stages (Stage A: Complaint Intake, Stage B: Review and Assignment, Stage C: Repair, Stage D: Closure) and eight data stores (D1–D8).

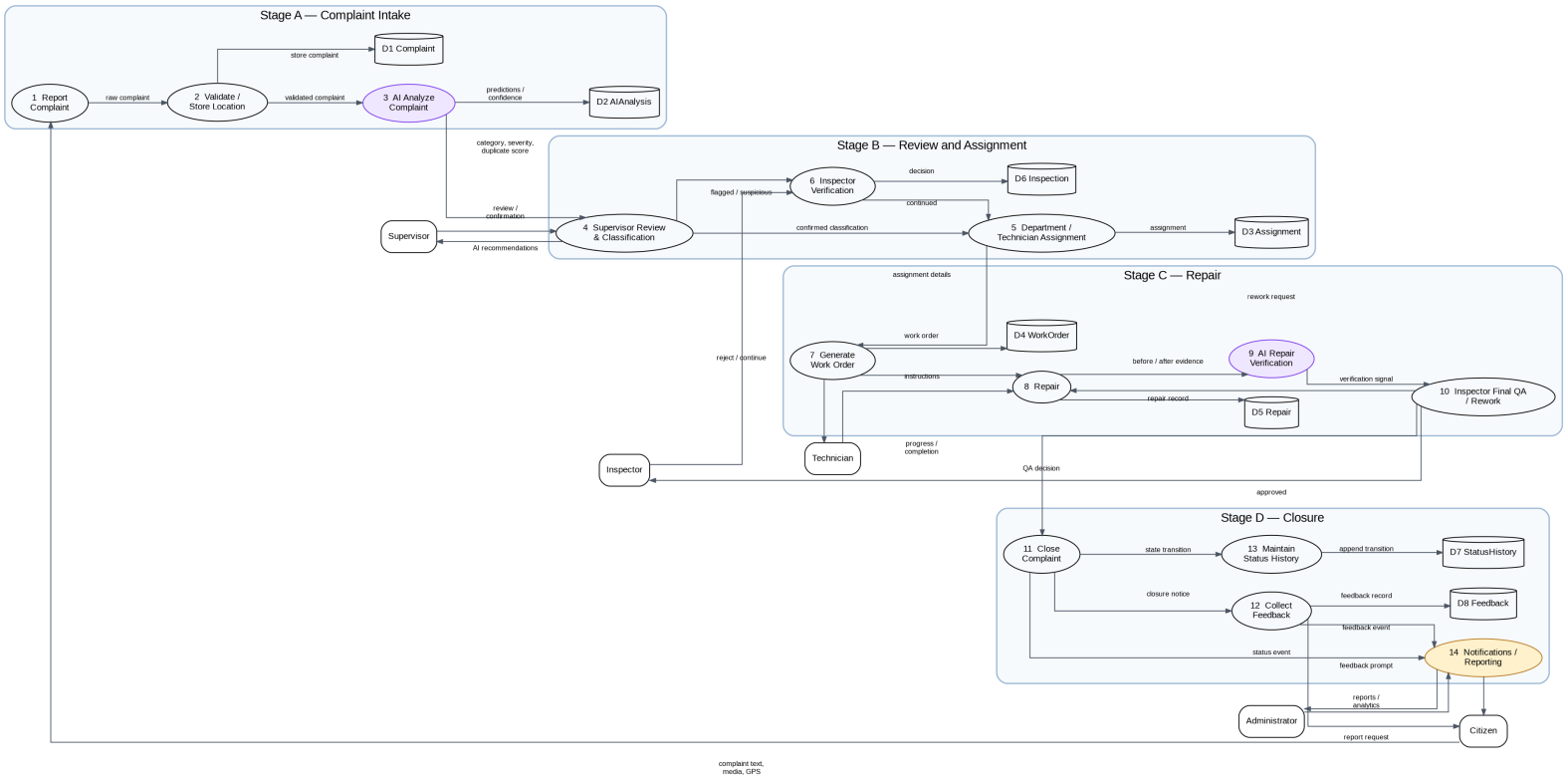


Figure 2.4: DFD Level 1 — four-stage detailed process flow guiding the data-flow implementation (Stage A: Complaint Intake, Stage B: Review and Assignment, Stage C: Repair, Stage D: Closure).

2.3.3 DFD Level 2

Figure 2.5 decomposes process 3 (AI Analyze Complaint) from Level 1 into the three core models. Duplicate detection reads existing complaints from D1; all predictions are written to D2 and reach the Supervisor only as recommendations.

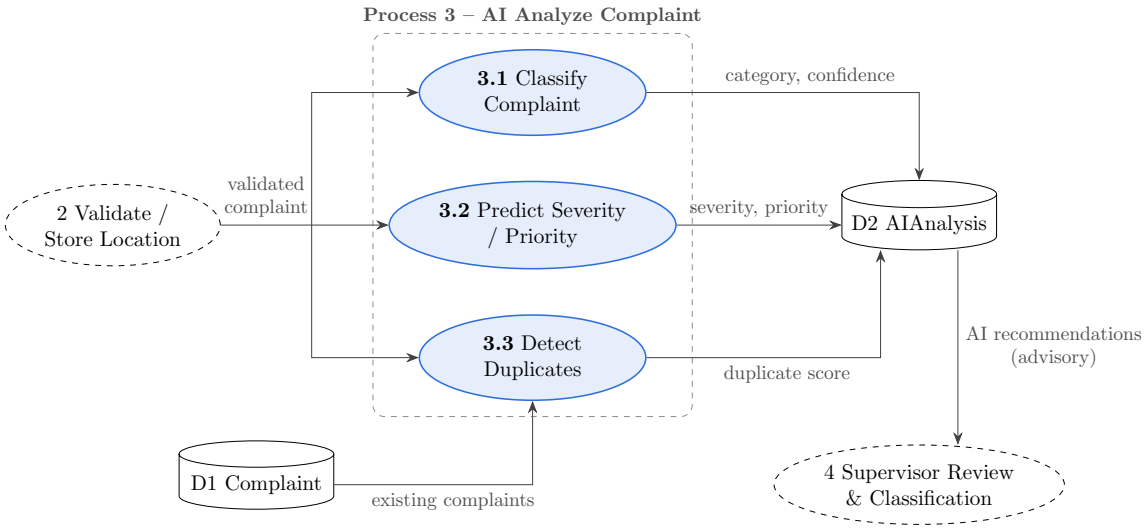


Figure 2.5: DFD Level 2 — decomposition of process 3, AI Analyze Complaint. Dashed ellipses are the neighbouring Level 1 processes.

2.4 Software Requirement Specification in IEEE Format

This section follows the structure of IEEE Std 830 [IEE98].

1. Introduction

1.1 Purpose

This SRS defines the requirements for the Public Infrastructure Issue Resolution System, a platform for reporting municipal infrastructure problems and tracking them to verified closure.

1.2 Scope

The system gives citizens one channel to report issues with text, media and GPS location. It moves each complaint through a role-based, auditable workflow. An

internal AI module supports triage with classification, severity/priority prediction and duplicate detection, but makes no final decisions.

1.3 Definitions and Acronyms

Term	Meaning
RBAC	Role-Based Access Control
JWT	JSON Web Token, used for authentication
DFD	Data Flow Diagram
WorkOrder	A repair task assigned to a Technician for one complaint
StatusHistory	Audit log of every complaint state change
AIAnalysis	Stored AI predictions for a complaint

Table 2.1: SRS definitions and acronyms

1.4 References

FastAPI [Ram26]; RFC 7519 [JBS15]; Role-Based Access Controls [FK92]; UML 2.5.1 [Obj17]; Sommerville [Som16]; Yourdon [You89].

2. Overall Description

2.1 Product Perspective

A standalone web application with a FastAPI backend, a relational database and an internal AI layer. It replaces manual registers, phone calls and per-department spreadsheets.

2.2 Product Functions

- Register, log in and manage profile.
- Submit a complaint with evidence and location; track its status.
- AI classification, severity/priority prediction and duplicate detection.
- Supervisor review, classification, assignment, reassignment and escalation.
- Technician work updates and before/after evidence upload.
- Inspector verification, rework requests and closure.
- Citizen feedback on resolved complaints.
- Notifications for submissions, updates, mentions and alerts.
- Administration of users, roles, departments, categories, settings and reports.

2.3 User Classes

Role	Responsibility
Citizen	Submits, tracks and gives feedback on complaints
Supervisor	Reviews AI output; classifies, assigns, reassigns and escalates
Technician	Carries out assigned repairs and uploads evidence
Inspector	Verifies completed work, requests rework, closes complaints
Administrator	Manages users, roles, departments, categories and reports

Table 2.2: SRS user classes

2.4 Operating Environment

Web browser client; FastAPI server; relational database.

2.5 Design Constraints

AI output is advisory only. Every state change must be recorded in `StatusHistory`. Access is controlled by JWT authentication and RBAC.

2.6 Assumptions and Dependencies

Citizens have a device that can capture photos/video and share GPS location. AI model accuracy depends on the training data available.

3. Specific Requirements

3.1 Functional Requirements

ID	Requirement
FR-01	The system shall let users register and log in with email and password.
FR-02	The system shall restrict each feature to the user's role through RBAC.
FR-03	The system shall let a Citizen submit a complaint with title, description, photo/video evidence and GPS location.
FR-04	The system shall validate and store each complaint with status `New`.
FR-05	The AI module shall predict a category for each complaint.

ID	Requirement
FR-06	The AI module shall predict severity (Low, Medium, High, Critical) and a priority score.
FR-07	The AI module shall flag likely duplicates of existing complaints with a duplicate score.
FR-08	The system shall show AI predictions to the Supervisor as recommendations only.
FR-09	The Supervisor shall be able to confirm or correct classification, assign a Department and Technician, reassign and escalate.
FR-10	The system shall create a `WorkOrder` on assignment.
FR-11	The Technician shall be able to update `WorkOrder` status and upload before/after evidence.
FR-12	The Inspector shall be able to approve work, request rework, and close complaints.
FR-13	The system shall record every status change in `StatusHistory` with the user or workflow event that caused it.
FR-14	The system shall notify users of submissions, updates, mentions and alerts.
FR-15	The Citizen shall be able to track complaint status and rate a resolved complaint from 1 to 5 with comments.
FR-16	The Administrator shall be able to manage users, roles, departments, categories and settings, and view reports.

Table 2.3: Functional requirements

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	Passwords shall be stored as hashes; sessions shall use JWT.
NFR-02	Security	Only the permitted role may perform each workflow action.
NFR-03	Auditability	No complaint state change shall occur without a `StatusHistory` entry.
NFR-04	Accountability	AI shall never autonomously approve, reject, assign or close a complaint.
NFR-05	Usability	Citizens shall be able to submit a complaint through a simple form with media and location.
NFR-06	Maintainability	Backend, database and AI layer shall be separate modules.

Table 2.4: Non-functional requirements

3.3 External Interface Requirements

- **User interface:** web dashboards per role (Citizen, Supervisor, Technician, Inspector, Administrator).
- **Software interface:** REST endpoints in FastAPI for complaint creation, review, assignment and status updates; relational database for persistence.
- **Hardware interface:** device camera for evidence and GPS for location.

2.5 User Stories and Story Cards

The ten story cards below cover all five roles. Each card states who wants the feature, what they want and why, followed by its acceptance criteria.

US-01 **Citizen**

As a citizen, I want to report an issue with a photo and my location, so the right team can find and fix it.

Acceptance: Form accepts title, description, photo/video and GPS; complaint is saved with status `New`; I get a confirmation notification.

US-02 **Citizen**

As a citizen, I want to track my complaint, so I know it was received and is being worked on.

Acceptance: I can see the current status (`New`, `Assigned`, `InProgress`, `PendingVerification`, `Resolved`, `Closed`) and get a notification on each change.

US-03 **Citizen**

As a citizen, I want to rate the resolution, so the city knows whether the fix worked.

Acceptance: I can rate 1 to 5 and comment, only after the complaint is resolved.

US-04 **Supervisor**

As a supervisor, I want to see the AI's suggested category and priority, so I can triage faster.

Acceptance: Category, severity, priority score and confidence are shown; I can accept or change them.

US-05 **Supervisor**

As a supervisor, I want to be warned about duplicate complaints, so the same issue is not fixed twice.

Acceptance: Likely duplicates show a duplicate score and a link to the original; I decide whether to merge.

US-06 **Supervisor**

As a supervisor, I want to assign, reassign or escalate a complaint, so it reaches the right department.

Acceptance: Assigning creates a `WorkOrder`, notifies the technician, and logs the change in `StatusHistory`.

US-07 **Technician**

As a technician, I want to see my assigned work and update its progress, so supervisors know where it stands.

Acceptance: I see only my work orders and can set `InProgress` or `Completed` with notes.

US-08 **Technician**

As a technician, I want to upload before and after photos, so my repair can be verified.

Acceptance: Evidence is stored against the complaint with type, time and uploader.

US-09 **Inspector**

As an inspector, I want to approve or reject completed work, so only real fixes get closed.

Acceptance: Approve sets `Resolved`; reject sets `ReworkRequested` and returns it to the technician; I can close a resolved complaint.

US-10 **Administrator**

As an administrator, I want to manage users, roles, departments and categories, so the system matches the city's structure.

Acceptance: I can add, edit and deactivate users, change roles, manage departments and categories, and view reports.

Chapter 3

Design and Implementation

3.1 System Architecture

3.1.1 High-Level Design

The system follows a role-based workflow spanning four stages — complaint intake, review and assignment, repair, and closure — driven by five user roles (Citizen, Supervisor, Technician, Inspector, and Administrator) and a relational data model of Users, Complaints, Departments, WorkOrders, and their supporting records. Figure 3.1 shows the core entity relationships, modelled using standard UML class-diagram notation [Obj17].

3.1.2 Technical Stack

- **Backend (planned):** FastAPI [Ram26], with JWT-based authentication [JBS15] and RBAC middleware [FK92] for role separation between Supervisor, Technician, Inspector, and Administrator.
- **Database:** A relational database is connected and actively storing application data — the persistence layer for complaints, users, and workflow records.
- **Frontend:** An initial dashboard shell is in place as the working frontend surface, to be extended with role-specific views.
- **AI layer (planned):** Three advisory models — complaint classification, severity/priority prediction, and duplicate complaint detection — feeding recommendations into the review stage without autonomous decision authority.

3.2 Implementation Details

At the current stage, the project's foundational setup work is complete: the login page is functional, the database is connected and actively storing application data, a basic dashboard shell is in place, and the system's UML design diagrams have been produced. The larger share of implementation — backend workflow logic, the complaint data-flow pipeline, AI model integration, and full role dashboards — remains ahead.

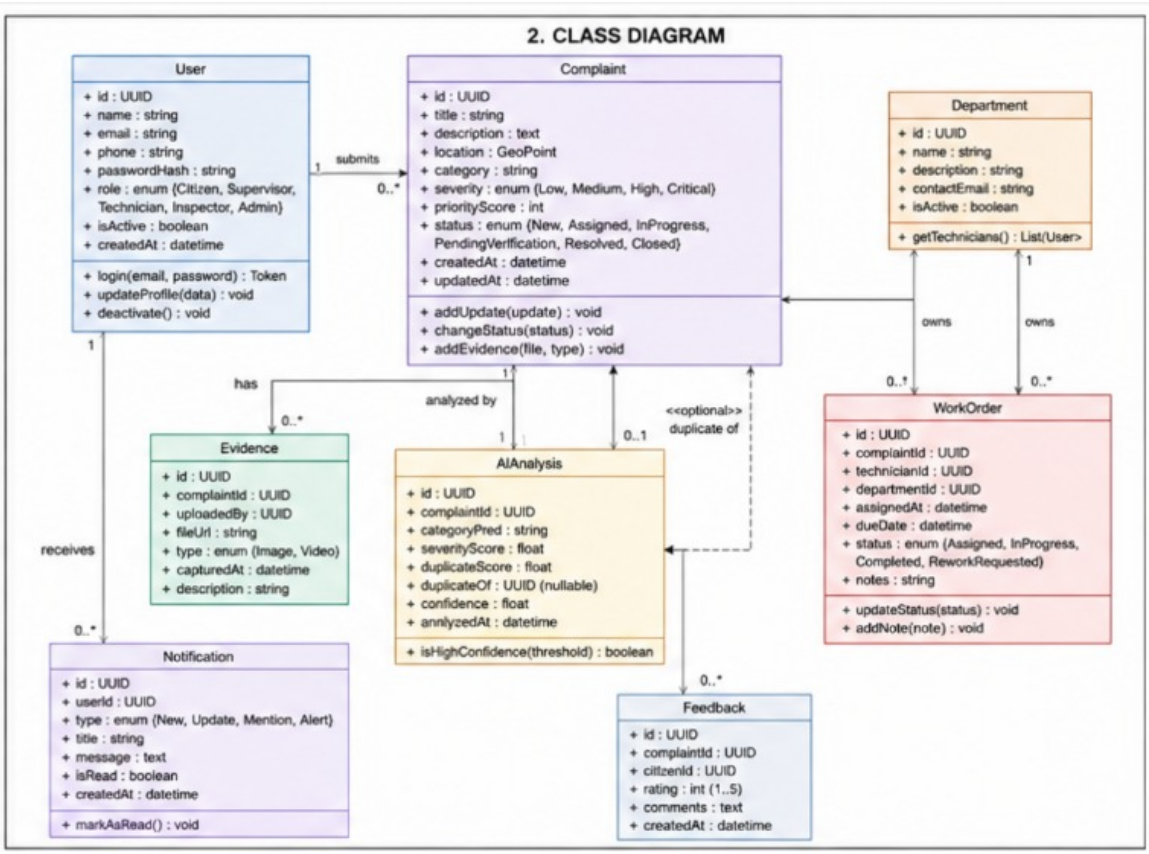


Figure 3.1: Class diagram covering the core system entities. A User submits Complaints and receives Notifications; each Complaint accumulates Evidence, is analysed by an AIAnalysis record (which may flag it as a duplicate of another Complaint), and is fulfilled through a WorkOrder owned by a Department; Citizens may also leave Feedback on a resolved Complaint.

Module	Weight	Complete	Status
Login / Authentication Page	10%	100%	Done
Database Connectivity & Storage	15%	100%	Done
Basic Dashboard (initial shell)	10%	100%	Done
UML / SE Design Diagrams	10%	100%	Done
Backend API & Business Logic	15%	0%	Pending
Complaint Data-Flow Implementation	10%	0%	Pending
AI Model Integration (3 core models)	20%	0%	Pending
Full Role Dashboards & UI Polish	5%	0%	Pending
Testing & Deployment	5%	0%	Pending
Overall	**100%**	**45%**	

Table 3.1: Module-wise completion weighting

3.2.1 Completed Components

- **Login Page:** The citizen and staff login/authentication page has been built and is functional, providing the entry point through which users will access their respective role-based views.
- **Database Connectivity & Storage:** The database has been connected to the application and is actively storing data, establishing the persistence layer that the rest of the system will be built on.
- **Basic Dashboard:** An initial dashboard shell has been put together, giving the project a working frontend surface to extend as further roles and features are added.
- **UML and Analysis Diagrams:** The use case, class, swimlane and data-flow diagrams have been produced as part of the software-engineering design work (Figures 2.1 to 3.1).

3.2.2 Remaining Components

- **Backend API & Business Logic:** The FastAPI backend (endpoints for complaint creation, review, assignment, and status updates, along with JWT authentication and RBAC middleware) is the next major piece to build on top of the working database and login page.

- **Complaint Data-Flow Implementation:** The end-to-end data-flow logic (complaint intake, validation, and routing between the Complaint, Assignment, WorkOrder, and StatusHistory tables) described in the DFD Level 0, 1 and 2 designs still needs to be implemented (Figures 2.3–2.5).
- **AI Model Integration (3 core models):** Complaint classification, severity/priority prediction, and duplicate complaint detection are not yet built or integrated.
- **Full Role Dashboards & Testing:** Complete Supervisor, Technician, Inspector, and Administrator views are still needed, along with formal testing (unit, integration, RBAC, workflow) and final deployment.

Chapter 4

Results and Evaluation

4.1 Testing Methodology

Testing at this stage has been limited to the pieces already built:

- **Login Testing:** Authentication flow has been checked manually against valid and invalid credentials.
- **Database Testing:** Connectivity and basic read/write operations have been verified.
- **Remaining:** Unit, integration, RBAC, and workflow/state-transition testing, following standard software testing practice [Som16], are deferred until the back-end and data-flow logic are implemented.

4.2 Performance Metrics

Component	Target	Current
Core system modules complete	9	4
Overall project completion	100%	45%
Core AI models integrated	3	0

Table 4.1: Progress snapshot at the current evaluation point

4.3 Analysis

The project has a working foundation: a functioning login page, a database that is connected and actively storing data, an initial dashboard shell, and the UML design work needed to guide implementation. The citizen-facing reporting objective is partially met — the entry point exists, but complaint submission and tracking logic have not yet been built on top of it. The largest remaining risk is sequencing: backend logic and data-flow implementation need to land before AI integration can be meaningfully tested against real workflow data.

Chapter 5

Conclusion

5.1 Summary of Work

The project currently stands at approximately 45% complete. The login page, database connectivity and storage, a basic dashboard, and the analysis and design artefacts — use case, swimlane, DFD Level 0–2 and class diagrams, use case templates, the SRS and user stories — are in place, giving the team a working base and a clear design to build the remaining backend, data-flow, and AI components against.

5.2 Key Findings

- **Finding 1:** Setting up the database connection and the login flow early has reduced the risk for the rest of the build, since every remaining feature depends on both.
- **Finding 2:** Producing the UML and DFD diagrams ahead of the backend implementation gives the team a concrete reference for the entities and relationships still to be coded.
- **Finding 3:** AI integration remains the single largest piece of remaining work, consistent with the original feasibility plan.

5.3 Future Work and Recommendations

1. Build out the FastAPI backend — authentication middleware, RBAC, and core complaint endpoints.
2. Implement the complaint data-flow logic per the DFD Level 0, 1 and 2 designs.
3. Train and integrate the three core AI models (classification, severity/priority, duplicate detection).
4. Expand the basic dashboard into full role-specific views, then run the planned test suite.

5.4 Final Remarks

With the login page, database, basic dashboard, and the analysis and design work complete, the project has a solid and correctly sequenced foundation. The next milestone is the backend and data-flow implementation, which will unlock both AI integration and full dashboard development.

Bibliography

[FK92] David F. Ferraiolo and D. Richard Kuhn. “Role-Based Access Controls”. In: *Proceedings of the 15th National Computer Security Conference*. Baltimore, MD, 1992, pp. 554–563 (cit. on pp. 7, 18, 24).

[IEE98] IEEE. *IEEE Recommended Practice for Software Requirements Specifications*. IEEE Std 830-1998. IEEE Computer Society. 1998 (cit. on p. 17).

[JBS15] M. Jones, J. Bradley, and N. Sakimura. *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force, 2015 (cit. on pp. 7, 18, 24).

[Obj17] Object Management Group. *Unified Modeling Language (UML), Version 2.5.1*. OMG Document Number formal/2017-12-05. 2017 (cit. on pp. 10, 18, 24).

[Ram26] Sebastián Ramírez. *FastAPI*. 2026. URL: https://fastapi.tiangolo.com/ (visited on 09/08/2026) (cit. on pp. 7, 18, 24).

[Som16] Ian Sommerville. *Software Engineering*. 10th. Pearson, 2016 (cit. on pp. 18, 28).

[You89] Edward Yourdon. *Modern Structured Analysis*. Prentice-Hall, 1989 (cit. on pp. 13, 18).